

Arbres et Structures Hiérarchiques

420-311 Structures de données

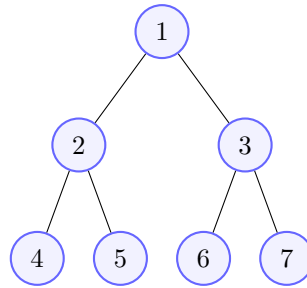
Septembre 2026

1 Introduction et Définitions

Un **arbre** est une structure de données hiérarchique non linéaire constituée d'un ensemble de nœuds connectés par des arêtes. C'est un cas particulier et contraint de graphe :

- C'est un graphe connexe et acyclique (sans boucle).
- Il possède un nœud unique désigné comme la **racine** (*root*).
- Chaque nœud (sauf la racine) possède un unique parent et un nombre quelconque de nœuds enfants. Les nœuds sans enfants sont appelés des **feuilles** (*leaves*).

1.1 Exemple d'un Arbre Binaire



2 Arbres Binaires et Efficacité ($\log n$)

Un **arbre binaire** est un arbre où chaque nœud possède au maximum deux enfants, généralement appelés *gauche* (*left*) et *droit* (*right*).

Pourquoi utiliser des arbres ? Dans une structure linéaire (tableau, liste chaînée), la recherche d'un élément prend un temps linéaire $O(n)$ dans le pire des cas. Dans un arbre binaire de recherche équilibré, chaque niveau divise l'espace de recherche par deux, ce qui ramène la complexité de recherche à $O(\log n)$.

3 Stratégies de Parcours (Traversals)

Contrairement aux listes ou aux tableaux, il existe plusieurs façons de parcourir systématiquement tous les nœuds d'un arbre :

- **Pré-ordre (*Pre-order*)** : Traiter la racine → Explorer le sous-arbre gauche → Explorer le sous-arbre droit. (Utile pour copier un arbre).
- **In-ordre (*In-order*)** : Explorer le sous-arbre gauche → Traiter la racine → Explorer le sous-arbre droit. (Pour les arbres binaires de recherche, cela permet de visiter les éléments par ordre croissant).
- **Post-ordre (*Post-order*)** : Explorer le sous-arbre gauche → Explorer le sous-arbre droit → Traiter la racine. (Utile pour supprimer un arbre ou évaluer des expressions arithmétiques).
- **Parcours en largeur (*BFS / Level-order*)** : Explorer l'arbre niveau par niveau de haut en bas et de gauche à droite, en utilisant une file (*queue*).

4 Implémentations et Algorithmes en Python

4.1 1. Manipulation : Inversion itérative d'un arbre (DFS avec Pile)

Plutôt que d'utiliser la récursivité classique, cette méthode utilise une pile explicite pour inverser les enfants gauches et droits de chaque nœud de manière itérative.

```
def invert_tree(root):
    if not root:
        return None

    stack = [root]
    while stack:
        node = stack.pop()
        # change des enfants gauche et droit
        node.left, node.right = node.right, node.left

        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)

    return root
```

4.2 2. Algorithme concret : Évaluation d'un Arbre d'Expression (Post-ordre récursif)

Les arbres d'expression permettent de représenter des calculs mathématiques où les feuilles contiennent des nombres et les nœuds internes contiennent des opérateurs.

```
def evaluate_expression_tree(root):
    if not root:
        return 0

    # Si c'est une feuille, retourner sa valeur numérique
    if not root.left and not root.right:
        return int(root.val)

    # valuation récursive des sous-arbres (parcours post-ordre implicite)
    left_val = evaluate_expression_tree(root.left)
    right_val = evaluate_expression_tree(root.right)

    # Application de l'opérateur du nœud courant
    if root.val == '+': return left_val + right_val
    elif root.val == '-': return left_val - right_val
    elif root.val == '*': return left_val * right_val
    elif root.val == '/': return left_val / right_val
```

5 Applications des Arbres

- **Systèmes de fichiers** : Les dossiers et sous-dossiers de votre système d'exploitation (Windows, Linux, macOS) sont organisés sous forme d'arborescence.
- **Bases de données (Indexation B-Trees / B+ Trees)** : Permettent aux SGBD de retrouver des enregistrements sur disque en évitant des scans de tables complets.
- **Compilateurs (AST - Abstract Syntax Trees)** : Les langages de programmation sont convertis en arbres syntaxiques par les compilateurs pour analyser et optimiser le code.

- **Compression de données (Codage de Huffman)** : Utilise des arbres binaires basés sur les fréquences d'apparition des caractères (utilisé dans ZIP, JPEG).
- **Intelligence Artificielle** : Arbres de décision et arbres de recherche de jeux (ex. échecs, minimax).